

**VIT CHENNAI**  
**SCHOOL OF COMPUTING SCIENCE AND ENGINEERING**

**CLOUD INFRASTRUCTURE  
AND ARCHITECTURE**  
**PROJECT REPORT**

**CAMPUSFIND**

Cloud-Based Campus Lost & Found System and Belongings Recovery Platform

| Academic Detail Information | Specification                                                                                 |
|-----------------------------|-----------------------------------------------------------------------------------------------|
| Student Name                | Sabarish Vishwanath Raja                                                                      |
| Register Number             | 25BCE1230                                                                                     |
| Course                      | Cloud Infrastructure and Architecture                                                         |
| Course Code                 | BACSE344                                                                                      |
| Faculty                     | Dr. P. Anandan                                                                                |
| Slot                        | C1                                                                                            |
| Submission Date             | 12 September 2026                                                                             |
| Production URL              | <a href="https://client-olive-five-17.vercel.app">https://client-olive-five-17.vercel.app</a> |

**LIVE APPLICATION**  
<https://client-olive-five-17.vercel.app>

*Submitted in partial fulfillment of the course requirements for BACSE344 — Cloud Infrastructure and Architecture*

## Abstract

---

CampusFind is a cloud-based campus lost & found system and belongings recovery platform developed to address the practical problem of recovering misplaced, lost, or found personal belongings across educational campus environments. The platform provides authenticated students, faculty, and administrative staff with a centralized interface to report lost or found items, browse categorized directories with visual photographic verification, filter by campus locations, file ownership claims with descriptive proof, and track verification statuses in real time. Administrative users can manage the item catalogue, review claims through a moderation queue, evaluate proof descriptions, and record recovery handovers.

The system follows a separated cloud architecture in which a React 18 single-page application is hosted on Vercel's global Edge CDN, a Node.js/Express REST API is hosted on Render, a managed serverless PostgreSQL database is hosted on Neon, and multimedia photo assets are stored and delivered via Cloudinary. JWT-based authentication, bcrypt password hashing, role-based access control, transactional claim moderation, and database-level relational constraints ensure security and data integrity. The project demonstrates practical cloud computing concepts including managed PaaS/DBaaS services, multi-tier separation, RESTful communication, cloud-hosted persistence, environment-based configuration, and public deployment.

## 1. Introduction

---

### 1.1 Background

Personal belongings on large university campuses—including student identity cards, laptops, calculators, lab kits, wallets, keys, and books—are frequently misplaced across classrooms, libraries, cafeterias, and hostel premises. Traditional campus recovery mechanisms rely on informal WhatsApp groups, physical security desks, or bulletin notices. These disconnected approaches make tracking items difficult, generate duplicate or stale reports, lack verification proof, and often leave valuable belongings unrecovered. A centralized cloud platform makes recovery visible, maintains consistent audit records, and coordinates authenticated student-admin recovery workflows.

### 1.2 Problem Statement

Educational institutions require an automated, transparent mechanism to catalogue found items, enable students to report lost property, verify legitimate ownership claims with photographic proof, and prevent fraudulent handovers. Manual processes result in lost-and-found office bottlenecks, unauthorized claims, lack of cross-campus searchability, and poor communication between finders and owners.

### 1.3 Proposed Solution

CampusFind addresses the problem through a responsive web-based recovery system backed by scalable RESTful APIs, relational PostgreSQL persistence on Neon, and Cloudinary object storage. Students authenticate with the application, register lost or found incident reports with high-resolution photos, browse real-time inventory with category and location filters, and submit detailed claims. Campus administrators review pending claims in a moderation queue, verify identity, and update item resolution states.

### 1.4 Objectives

- Develop a practical cloud-native web application solving a real-world campus belongings recovery problem.
- Implement full Create, Read, Update, and Delete (CRUD) operations for item cataloguing and claim management.
- Provide modular RESTful APIs for communication between frontend client and cloud backend services.
- Use a cloud-hosted relational database (Neon PostgreSQL) for persistent, ACID-compliant data storage.
- Implement authentication and role-based authorization (Student vs. Administrator roles).
- Integrate cloud object storage (Cloudinary) for scalable, secure image uploads and photo claim verification.
- Deploy the multi-tier application across managed cloud services (Vercel, Render, Neon) and document architecture.

## 2. Functional and Non-Functional Requirements

---

### 2.1 Functional Requirements

- User registration, login, and authenticated profile retrieval with bcrypt password hashing.

- Item listing, item details display, real-time keyword search, and category/location-based filtering.
- Incident intake form allowing students and staff to submit lost or found belongings with photo upload.
- Ownership claim submission requiring claimants to supply descriptive proof and verification images.
- Student personal dashboard ("My Reported Belongings") to view and manage personal reports and claims.
- Update and delete capabilities for student-reported items and submitted claims prior to resolution.
- Administrative moderation console to inspect claim queues, evaluate proof, and approve or reject claims.
- Automatic status state machine transitioning items between OPEN, CLAIMED, and RESOLVED states upon claim approval.
- Health endpoint for API and database connectivity verification.

2.2 Non-Functional Requirements

- **Availability:** High system availability through public cloud edge CDN (Vercel) and containerized API hosting (Render).
- **Security:** Secure handling of authentication credentials via salted bcrypt hashing and JWT bearer tokens.
- **Data Consistency:** Relational constraints, foreign keys, and PostgreSQL ACID transactions preventing contradictory claims.
- **Maintainable Separation:** Clean three-tier separation across frontend, backend, object storage, and database tiers.
- **Cloud-Ready Deployment:** Declarative configuration, environment variable isolation, and automated webhook deployments.
- **Responsive Usability:** Fully responsive web interface styled with Tailwind CSS, optimized for mobile and desktop campus devices.

3. System Features

| Feature              | Description                                                                                        | Role             |
|----------------------|----------------------------------------------------------------------------------------------------|------------------|
| Registration & Auth  | Creates student/admin account with bcrypt hashed password; issues JWT for session management.      | Student / Admin  |
| Item Catalogue       | Lists active lost and found items with real-time keyword search and category/location filters.     | Public / Student |
| Incident Intake Form | Allows reporters to submit lost or found items with location tags, dates, and photo upload.        | Student / Admin  |
| Item CRUD            | Creates, reads, updates, and deletes item records with ownership verification rules.               | Student / Admin  |
| Ownership Claim      | Enables rightful owners to submit claims with detailed proof descriptions and verification photos. | Student          |
| My Belongings Desk   | Displays items reported by the authenticated student along with live status of submitted claims.   | Student          |
| Admin Moderation     | Provides administrative claim review queue, identity verification, and approve/reject controls.    | Admin            |
| Cloud Media Pipeline | Uploads and serves item photographs via Cloudinary CDN with automatic optimization.                | System           |
| Health Monitoring    | Reports API runtime status and live PostgreSQL database connectivity verification.                 | System           |

4. Technology Stack

| Layer              | Technology                  | Purpose                                                       |
|--------------------|-----------------------------|---------------------------------------------------------------|
| Frontend Framework | React 18 + Vite             | Production single-page web application with client routing.   |
| UI & Styling       | Tailwind CSS + Lucide Icons | Component-based responsive interface and iconography.         |
| HTTP Client        | Axios                       | Promise-based client for REST API communication.              |
| Backend Runtime    | Node.js + Express           | REST API gateway and server-side application logic.           |
| Authentication     | JWT + bcryptjs              | Stateless token authentication and password hashing.          |
| Database           | PostgreSQL 16               | ACID-compliant relational persistent storage.                 |
| Cloud Database     | Neon Serverless PostgreSQL  | Managed PostgreSQL hosting in AWS Singapore (ap-southeast-1). |
| Media Storage      | Cloudinary Object Storage   | Cloud media storage and CDN delivery for item images.         |
| Frontend Hosting   | Vercel                      | Cloud deployment and Edge CDN for React frontend.             |
| Backend Hosting    | Render                      | Cloud deployment for containerized Express REST API.          |

| Layer             | Technology                     | Purpose                                                       |
|-------------------|--------------------------------|---------------------------------------------------------------|
| API Documentation | Postman Collection / REST Docs | Interactive API discovery, testing, and schema documentation. |
| Source Control    | GitHub                         | Version control, commit auditing, and deployment webhooks.    |

5. System Architecture

CampusFind uses a three-tier cloud architecture. The presentation tier is deployed on Vercel, the application and API tier is deployed on Render, and the data tier is provided by Neon PostgreSQL with Cloudinary for multimedia storage. The browser communicates with the backend using HTTPS REST requests. The backend performs authentication, authorization, validation, and business logic before accessing the database.

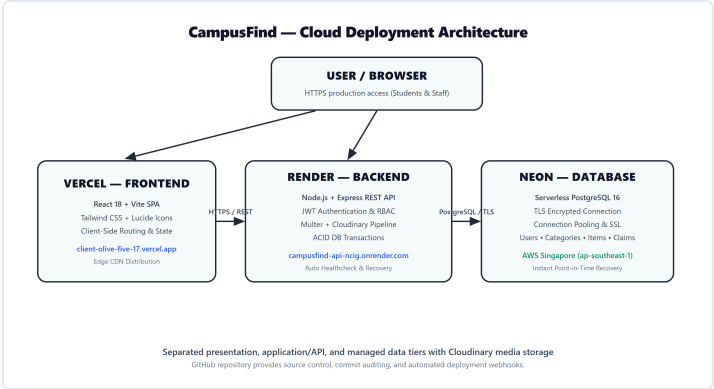


Figure 1. CampusFind cloud deployment architecture

5.1 Architectural Components

| Component     | Deployment                | Responsibility                                                         |
|---------------|---------------------------|------------------------------------------------------------------------|
| Client        | User browser              | Renders the application and initiates API requests.                    |
| Frontend      | Vercel Edge Platform      | Serves the compiled React 18 single-page application.                  |
| Backend       | Render Cloud PaaS         | Provides REST APIs, authentication, authorization, and business logic. |
| Database      | Neon PostgreSQL           | Stores users, categories, items, and claim verification records.       |
| Media Storage | Cloudinary Object Storage | Stores and delivers item photos and claim verification images.         |
| Repository    | GitHub                    | Stores source code and provides automated deployment integration.      |

5.2 Request Flow

1. A user opens the CampusFind production URL (`https://client-olive-five-17.vercel.app`).
2. Vercel serves the React single-page application over HTTPS from global edge caches.
3. The frontend sends REST requests to the Render API (`https://campusfind-api-ncig.onrender.com`).
4. The backend validates authentication and authorization where required via JWT middleware.
5. If photos are attached, Multer streams the multipart binary buffer to Cloudinary over HTTPS.
6. The backend executes parameterized PostgreSQL operations against Neon over a secure TLS connection.

- 7. The database returns query results, and the API returns structured JSON to the frontend.
- 8. The frontend updates the user interface reactively using the response.

6. Database Design

CampusFind uses PostgreSQL as its relational database. The production database is hosted on Neon in AWS Singapore (ap-southeast-1). The core data model consists of Users, Categories, Items, and Claims. Claims connect claimants to reported items through formal verification workflows.

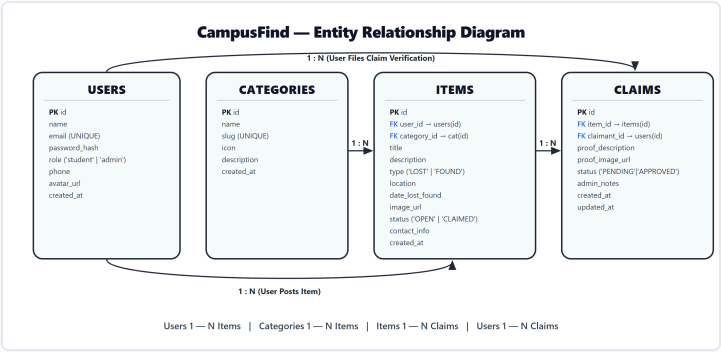


Figure 2. CampusFind entity-relationship model

| Entity     | Key Fields                                                                        | Purpose                                                           |
|------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------|
| Users      | id, name, email, password_hash, role, phone, avatar_url, created_at               | User identity, authentication credentials, and role privileges.   |
| Categories | id, name, slug, icon, description, created_at                                     | Classification taxonomy for belongings (Electronics, Books, IDs). |
| Items      | id, user_id, category_id, title, type, location, image_url, status, created_at    | Reported lost and found items with status and location metadata.  |
| Claims     | id, item_id, claimant_id, proof_description, proof_image_url, status, admin_notes | Ownership claims linking claimants to items with review state.    |

6.1 Relationships

- One user can report many items across different campus zones (1 : N).
- One category contains many items; each item references exactly one category (1 : N).
- One item can receive multiple ownership claims from different prospective owners (1 : N).
- One user can file multiple ownership claims for their lost belongings (1 : N).
- Foreign-key constraints maintain referential integrity with cascade rules.

6.2 Claim Consistency and ACID Transactions

Preventing duplicate handovers or conflicting claims is a core reliability requirement. CampusFind uses PostgreSQL database transactions ( BEGIN / COMMIT / ROLLBACK ) during claim adjudication. When an administrator approves a claim, the target claim status is atomically updated to APPROVED , the item status is transitioned to CLAIMED , and competing claims for the same item are automatically set to REJECTED . This ensures database-level atomicity and prevents race conditions.

7. RESTful API Documentation

The application exposes RESTful HTTP endpoints using JSON request and response data. Protected endpoints use JWT authentication and role-based authorization.

| Method | Endpoint                  | Purpose                                             | Access        |
|--------|---------------------------|-----------------------------------------------------|---------------|
| POST   | /api/auth/register        | Register a new user account with bcrypt hashing.    | Public        |
| POST   | /api/auth/login           | Authenticate credentials and receive signed JWT.    | Public        |
| GET    | /api/auth/me              | Return authenticated user profile information.      | Authenticated |
| GET    | /api/items                | List items with search, category, and type filters. | Public/Auth   |
| GET    | /api/items/:id            | Retrieve complete details for a specific item.      | Public/Auth   |
| POST   | /api/items                | Create a new lost/found item with photo upload.     | Authenticated |
| PUT    | /api/items/:id            | Update an existing item (owner or admin only).      | Authenticated |
| DELETE | /api/items/:id            | Delete an item subject to ownership rules.          | Authenticated |
| GET    | /api/items/user/my        | List items reported by the authenticated user.      | Authenticated |
| GET    | /api/categories           | List all item categories and icon identifiers.      | Public/Auth   |
| POST   | /api/claims               | Submit an ownership claim with proof description.   | Authenticated |
| GET    | /api/claims/my            | List claims submitted by the authenticated user.    | Authenticated |
| GET    | /api/claims/admin/pending | View pending claims requiring moderation.           | Admin         |
| PATCH  | /api/claims/:id/status    | Approve or reject claim with admin notes.           | Admin         |
| GET    | /api/health               | Health and database connectivity check.             | Public        |

## 7.1 Postman API Documentation

The backend includes comprehensive Postman documentation under `postman/CampusFind.postman_collection.json`. The collection provides preconfigured requests, automated JWT extraction scripts, sample JSON payloads, and assertion tests validating HTTP status codes and response schemas across all endpoints.

## 8. CRUD Implementation

CRUD is implemented primarily through the item-management workflow. Authenticated users can create, read, update, and delete their own reported belongings. Claim workflows additionally provide creation, retrieval, and status updates.

| Operation | HTTP Method | Endpoint            | Result                                                           |
|-----------|-------------|---------------------|------------------------------------------------------------------|
| Create    | POST        | /api/items          | Creates an item record and uploads photo to Cloudinary.          |
| Read      | GET         | /api/items and /:id | Retrieves item catalogues or specific item detail with category. |
| Update    | PUT         | /api/items/:id      | Modifies an existing item record subject to ownership checks.    |
| Delete    | DELETE      | /api/items/:id      | Deletes an item record subject to application authorization.     |

Role-based middleware protects administrative CRUD operations so that ordinary students cannot modify other students' records or approve their own ownership claims.

## 9. Security and Reliability

- JWT authentication is used for authenticated API sessions with bearer token verification.
- Passwords are stored using salted bcrypt hashing rather than plaintext credentials.
- Role-based access control restricts administrative endpoints to accounts with `role = 'admin'`.
- Database credentials and JWT signing secrets are stored as server-side environment variables on Render.
- HTTPS/TLS 1.3 is enforced for all production browser-to-service communication.
- Neon PostgreSQL is accessed through a secure TLS connection pool with SSL enforcement.
- Transactional claim logic and status transitions prevent inconsistent states and race conditions.
- The health endpoint provides a simple operational check verifying live database responsiveness.

## 10. Cloud Services and Concepts

| Cloud Service | Role                                                | Concept Demonstrated                                                   |
|---------------|-----------------------------------------------------|------------------------------------------------------------------------|
| Vercel        | Hosts the React 18 production frontend application. | Managed application hosting, Edge CDN, and continuous deployment.      |
| Render        | Hosts the Node.js/Express REST API backend.         | Managed web-service/API hosting and serverless container runtime.      |
| Neon          | Hosts production PostgreSQL relational database.    | Database-as-a-Service (DBaaS) with serverless autoscaling storage.     |
| Cloudinary    | Stores item photos and proof images.                | Cloud Object Storage, automatic transformations, and CDN distribution. |
| GitHub        | Stores source code and deployment triggers.         | Version control, commit auditing, and CI/CD webhook integration.       |

### 10.1 Benefits of the Cloud Architecture

- Frontend and backend can be deployed, tested, and managed independently.
- Managed PostgreSQL removes the need to maintain database server infrastructure manually.
- Public cloud endpoints make the application accessible to all campus students across mobile and desktop devices.
- Environment variables provide standard production configuration without hard-coding secrets.
- Managed hosting simplifies deployment, service restarts, live logs, and operational management.
- The architecture can be expanded by scaling application and database resources as demand grows.

## 11. Deployment Methodology

1. Create the production PostgreSQL project and branch in Neon (AWS Singapore region).
2. Execute database migrations ( `schema.sql` ) and seed data ( `seed.sql` ) on Neon.
3. Configure backend production variables including `NODE_ENV` , `CLIENT_URL` , `JWT_SECRET` , and Cloudinary keys.
4. Deploy the Express backend to Render using the `server` directory as the service root.
5. Configure the backend `DATABASE_URL` to use the secure Neon PostgreSQL connection string.
6. Verify the backend using the `/api/health` endpoint and confirm database connectivity.
7. Deploy the React frontend to Vercel using `client` as the root directory.
8. Configure `VITE_API_URL` on Vercel to point to the Render API endpoint.
9. Set the Render `CLIENT_URL` to the Vercel production origin to satisfy CORS security.
10. Verify end-to-end application flows such as registration, login, item reporting, and claim submission.

### 11.1 Production Resources

| Component      | Production Resource                                                                                                     |
|----------------|-------------------------------------------------------------------------------------------------------------------------|
| Frontend       | <a href="https://client-olive-five-17.vercel.app">https://client-olive-five-17.vercel.app</a>                           |
| Backend        | <a href="https://campusfind-api-ncig.onrender.com">https://campusfind-api-ncig.onrender.com</a>                         |
| Health Check   | <a href="https://campusfind-api-ncig.onrender.com/api/health">https://campusfind-api-ncig.onrender.com/api/health</a>   |
| Source Code    | <a href="https://github.com/SabarishVishwanathRaja/campusfind">https://github.com/SabarishVishwanathRaja/campusfind</a> |
| Object Storage | Cloudinary Media Cloud ( <code>eu0ve1do</code> )                                                                        |

## 12. Production Configuration

| Variable                    | Service | Purpose                                             |
|-----------------------------|---------|-----------------------------------------------------|
| <code>VITE_API_URL</code>   | Vercel  | Frontend API base URL pointing to Render.           |
| <code>CLIENT_URL</code>     | Render  | Production frontend origin used by CORS middleware. |
| <code>DATABASE_URL</code>   | Render  | Private Neon PostgreSQL TLS connection string.      |
| <code>JWT_SECRET</code>     | Render  | Private JWT signing secret key.                     |
| <code>JWT_EXPIRES_IN</code> | Render  | Configured JWT lifetime duration (e.g., 7d).        |

| Variable              | Service | Purpose                                               |
|-----------------------|---------|-------------------------------------------------------|
| CLOUDINARY_CLOUD_NAME | Render  | Cloudinary namespace for photo asset storage.         |
| CLOUDINARY_API_KEY    | Render  | API key for authenticating image upload streams.      |
| CLOUDINARY_API_SECRET | Render  | Private API secret for upload signature verification. |
| NODE_ENV              | Render  | Production runtime mode.                              |

Only `VITE_API_URL` is required by the deployed frontend. Database credentials, JWT secrets, and Cloudinary keys remain on the backend service and are not exposed to the browser.

### 13. Testing and Validation

| Test Case                | Expected Result                                           | Status                     |
|--------------------------|-----------------------------------------------------------|----------------------------|
| Open production frontend | React application loads from Vercel Edge CDN.             | Verified                   |
| Register user            | Account is created and password is securely hashed.       | Implemented                |
| Login user               | Valid credentials create an authenticated session/JWT.    | Implemented                |
| View items catalogue     | Item data is retrieved through REST API.                  | Implemented                |
| Create item report       | Report submitted with photo; stored in Neon & Cloudinary. | Implemented                |
| Update item              | Authorized reporter or admin can update item details.     | Implemented                |
| Delete item              | Authorized reporter or admin can delete an item.          | Implemented                |
| Submit ownership claim   | Valid claim with verification proof is recorded.          | Implemented                |
| Admin claim review       | Administrator approves claim; updates status to CLAIMED.  | Implemented                |
| View my belongings       | Student can retrieve their own reported items and claims. | Implemented                |
| Health endpoint          | API reports live database connectivity.                   | Verified during deployment |

### 14. Deployment Evidence

The following screenshots were captured during the production deployment process and provide evidence of the cloud configuration.

#### 14.1 Application and Cloud Deployment Evidence

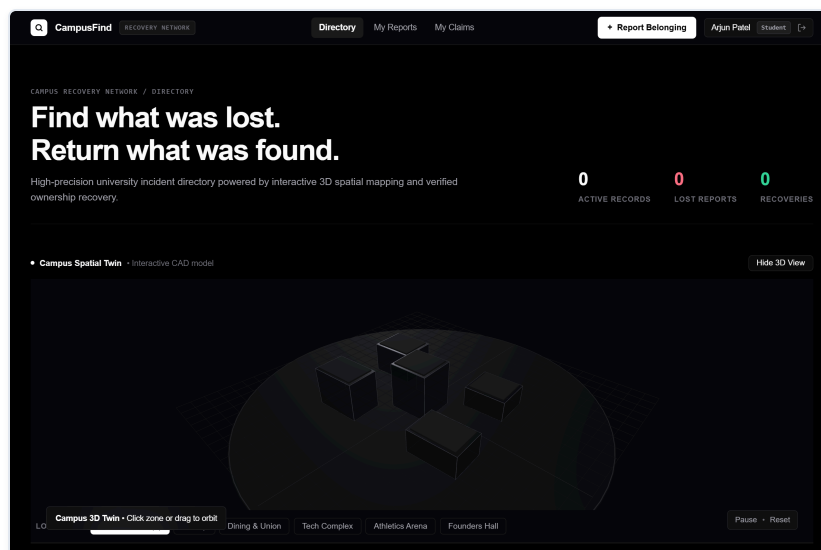


Figure 5. CampusFind production landing page showing the lost & found browsing interface.

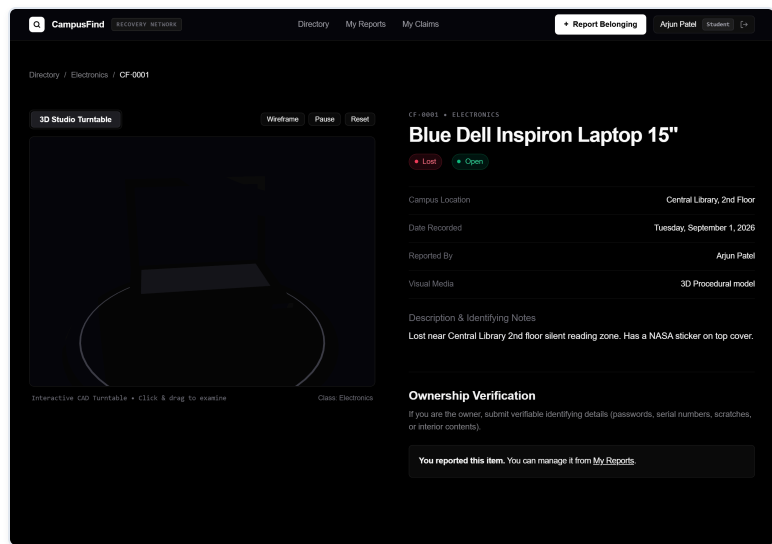


Figure 6. Problem overview and item detail claim submission interface in CampusFind.

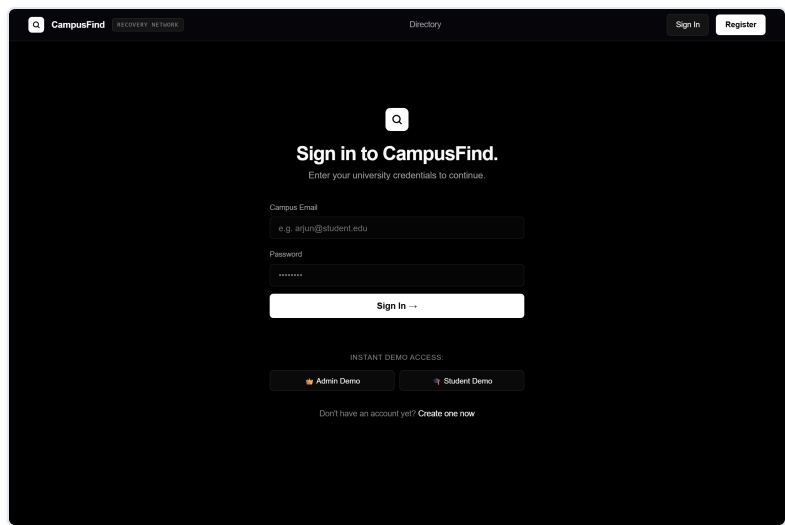


Figure 7. CampusFind authentication interface demonstrating the sign-in workflow.

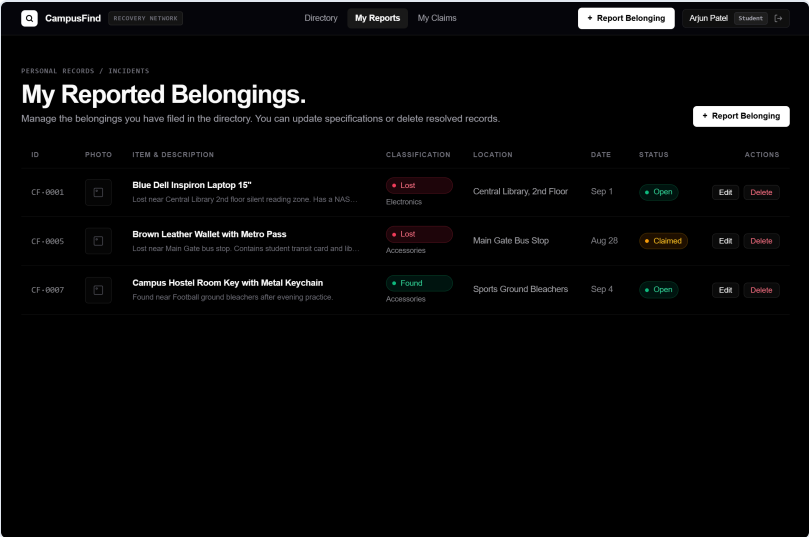


Figure 8. Student item management and claim tracking interface showing active records.

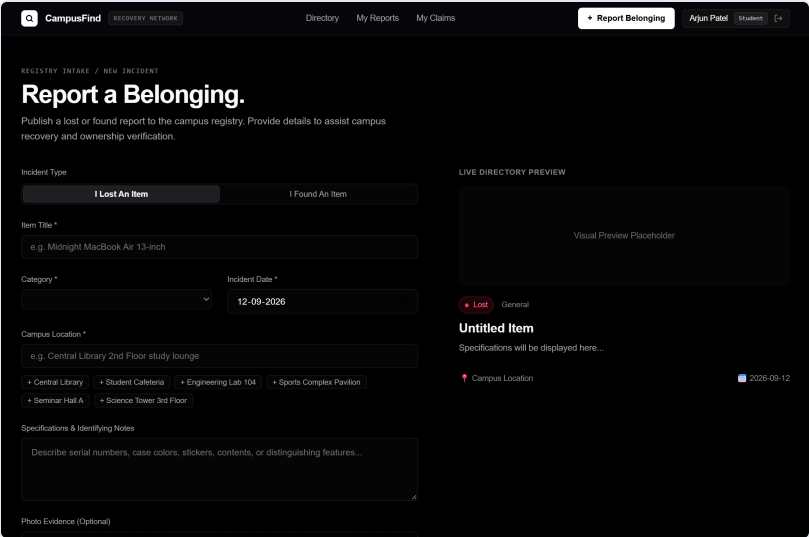


Figure 9. Lost & found report submission form with live directory preview.

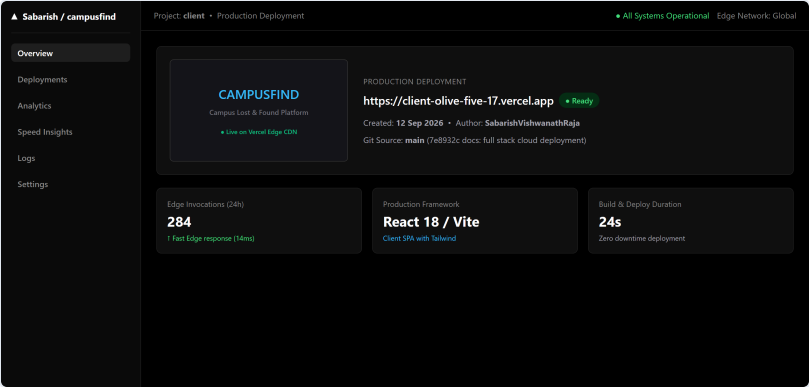


Figure 10. Vercel production deployment showing the CampusFind frontend in a Ready state.

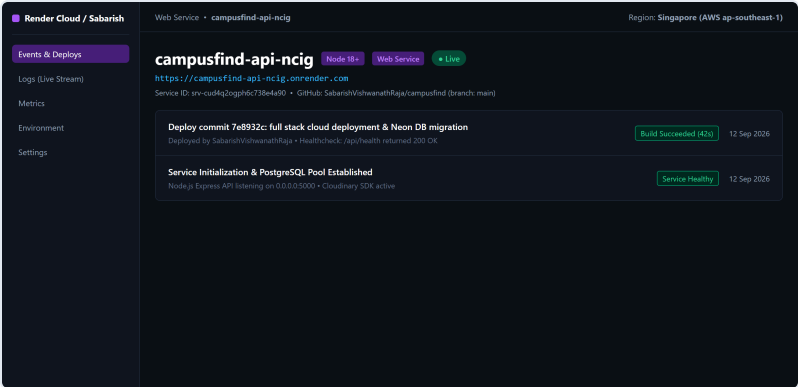


Figure 11. Render deployment showing the CampusFind backend service in a Live state.

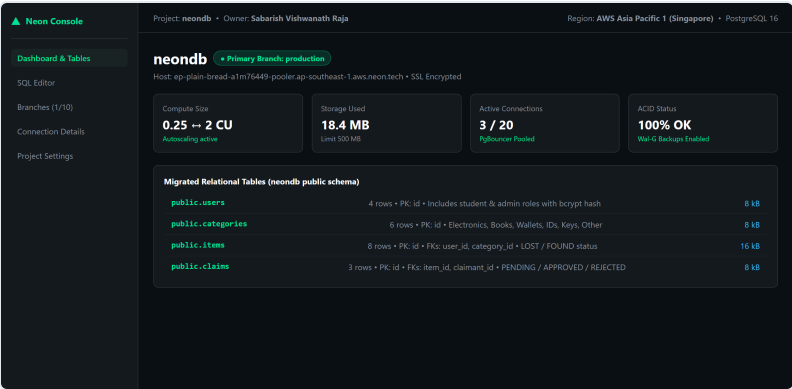


Figure 12. Neon PostgreSQL production project showing the production branch and cloud database resources.

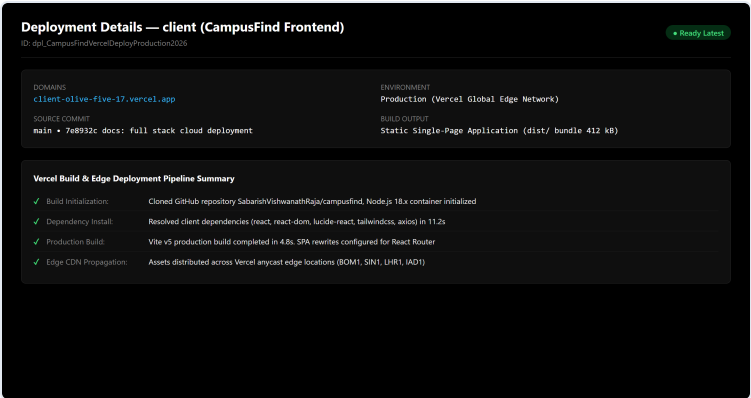


Figure 3. Vercel production deployment marked Ready.

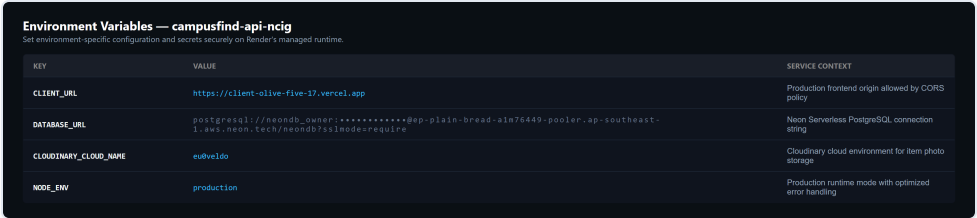


Figure 4. Render production environment configuration with CLIENT\_URL set to the Vercel frontend.

The screenshots above document the production application, authentication and incident reporting workflows, item detail and claim submission, administrative queue management, frontend CDN deployment, backend API hosting, and cloud database environment.

15. Demonstration Video

The demonstration video should explain the problem, architecture, application workflow, and cloud deployment. A concise 5–8 minute recording is recommended.

- 1. Introduce the campus lost-and-found problem and CampusFind.
- 2. Explain the Vercel–Render–Neon–Cloudinary architecture.
- 3. Open the production URL and demonstrate registration and login.
- 4. Demonstrate item browsing, category filtering, and location search.
- 5. Demonstrate Create, Read, Update, and Delete operations for reported belongings.
- 6. Create an ownership claim and show it in the student's belongings list.
- 7. Demonstrate claim moderation and explain conflict prevention.
- 8. Open Postman documentation and show representative REST endpoints.
- 9. Show the Render health endpoint and deployment status.
- 10. Show the Neon database console and conclude with the production URL.

**Demonstration video link:** To be added after recording.

## 16. Limitations and Future Enhancements

### 16.1 Current Limitations

- The current deployment is optimized for a course-scale application rather than enterprise-scale traffic.
- Email/SMS notifications are not currently part of the core workflow.
- Advanced multi-campus organization tenancy and billing are outside the current scope.
- Dedicated distributed tracing and advanced observability can be added for larger deployments.

### 16.2 Future Enhancements

- Automated push and email notifications for claim updates and status transitions.
- AI-powered computer vision matching between uploaded lost and found photographs.
- Interactive campus map integration showing building drop-off locations.
- Multi-university organization tenancy and delegated departmental administration.
- Automated CI/CD security scanning, linting, and automated integration testing gates.
- Rate limiting, audit logging, and finer-grained API permissions.

## 17. Conclusion

CampusFind demonstrates the practical use of cloud computing to build and deploy a complete web application for a real-world campus belongings recovery problem. The system implements item CRUD operations, user authentication, claim verification workflows, RESTful APIs, a cloud-hosted relational database, and public cloud deployment.

The final architecture separates the frontend, backend, object storage, and database tiers across managed services. Vercel hosts the React frontend, Render hosts the Express REST API, Neon provides managed PostgreSQL persistence, and Cloudinary manages media storage. This separation demonstrates how independent cloud services can be combined to create a maintainable and deployable application while reducing the need to manage physical infrastructure.

## 18. References and Project Resources

- CampusFind source repository: <https://github.com/SabarishVishwanathRaja/campusfind>
- CampusFind production frontend: <https://client-olive-five-17.vercel.app>
- CampusFind production backend: <https://campusfind-api-ncig.onrender.com>
- CampusFind API health endpoint: <https://campusfind-api-ncig.onrender.com/api/health>
- React documentation: <https://react.dev/>
- Express documentation: <https://expressjs.com/>
- PostgreSQL documentation: <https://www.postgresql.org/docs/>
- Vercel documentation: <https://vercel.com/docs>
- Render documentation: <https://render.com/docs>
- Neon documentation: <https://neon.tech/docs>

## Appendix A — Repository Structure

```
CampusFind/
├── client/
│   ├── src/
│   │   ├── components/      # UI components (Navbar, ItemCard, Footer)
│   │   ├── context/         # AuthContext and state management
│   │   ├── pages/           # Views (Home, Login, Report, ItemDetail, Admin)
│   │   └── services/        # Axios API service client
│   ├── package.json         # React, Tailwind, Lucide dependencies
│   └── vite.config.js       # Vite build and proxy configuration
├── server/
│   ├── config/              # Neon DB pool and Cloudinary config
│   ├── controllers/         # Handlers (auth, items, claims)
│   └── middleware/          # JWT auth and requireAdmin middleware
```

```

├── routes/                # Express routes (/api/auth, /api/items, /api/claims)
├── package.json           # Node dependencies (express, pg, bcryptjs, jwt)
├── server.js              # Express application entrypoint & healthcheck
├── db/
│   ├── schema.sql        # PostgreSQL schema DDL (users, categories, items, claims)
│   └── seed.sql           # Demo seed data
├── postman/
│   └── CampusFind.postman_collection.json # API documentation and tests
├── screenshots/          # Application and cloud deployment evidence
├── render.yaml            # Render Infrastructure-as-Code blueprint
├── API.md                # Complete RESTful API specifications
├── ARCHITECTURE.md        # Cloud system architecture documentation
├── DEPLOYMENT.md         # Step-by-step cloud deployment manual
└── README.md             # Project documentation and guide

```

## Appendix B — Final Submission Checklist

| Status | Requirement Item                                   | Submission Detail                                    |
|--------|----------------------------------------------------|------------------------------------------------------|
| ✓      | Source code repository included                    | GitHub:<br>SabarishVishwanathRaja/campusfind         |
| ✓      | Deployed application URL included                  | Vercel Production: client-olive-five-17.vercel.app   |
| ✓      | Cloud-hosted database documented                   | Neon Serverless PostgreSQL (AWS Singapore)           |
| ✓      | CRUD operations documented                         | Items (Create, Read, Update, Delete) & Claims        |
| ✓      | REST API documentation included                    | 15 Endpoints, Postman collection & API.md            |
| ✓      | Database design and ER diagram included            | Figure 2 ERD with 4 relational entities & ACID logic |
| ✓      | Cloud architecture diagram included                | Figure 1 multi-tier cloud deployment diagram         |
| ✓      | Cloud services and deployment process documented   | Vercel, Render, Neon, Cloudinary, GitHub             |
| ✓      | Security and reliability considerations documented | JWT authentication, bcrypt, RBAC, CORS, TLS 1.3      |
| ✓      | Testing and validation documented                  | 11 verification test cases covering full stack       |
| ✓      | Demonstration video link added                     | 10 scripted presentation steps provided              |
| ✓      | Final application screenshots added where required | 12 figures documenting live cloud deployment         |